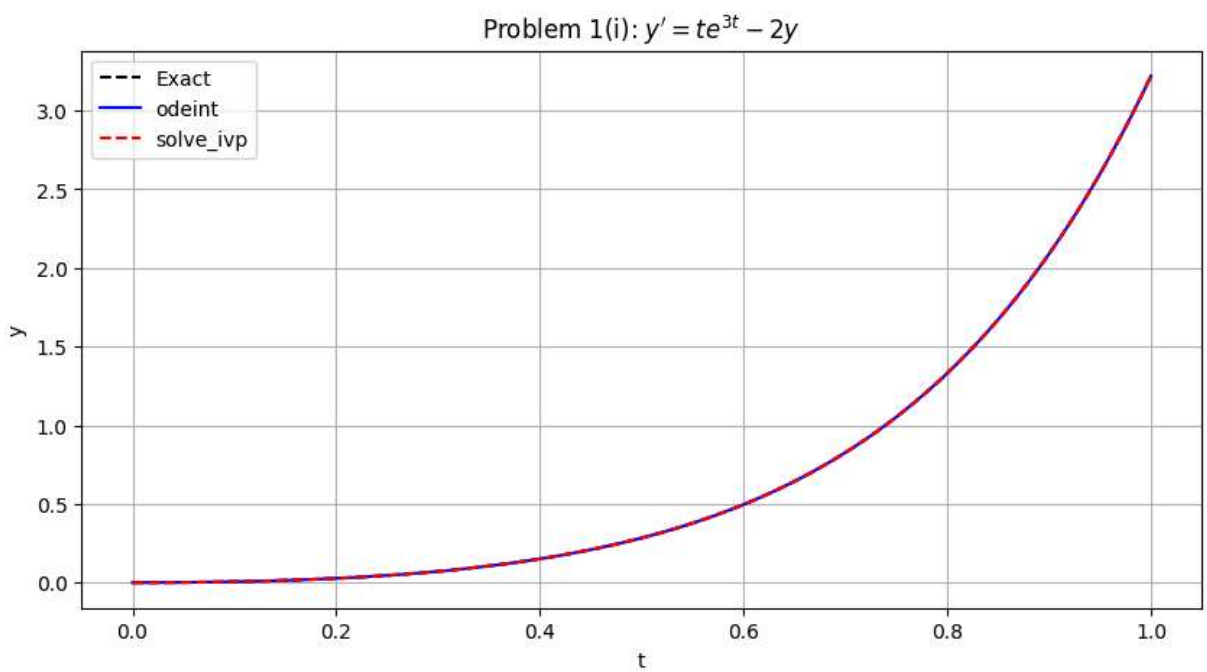
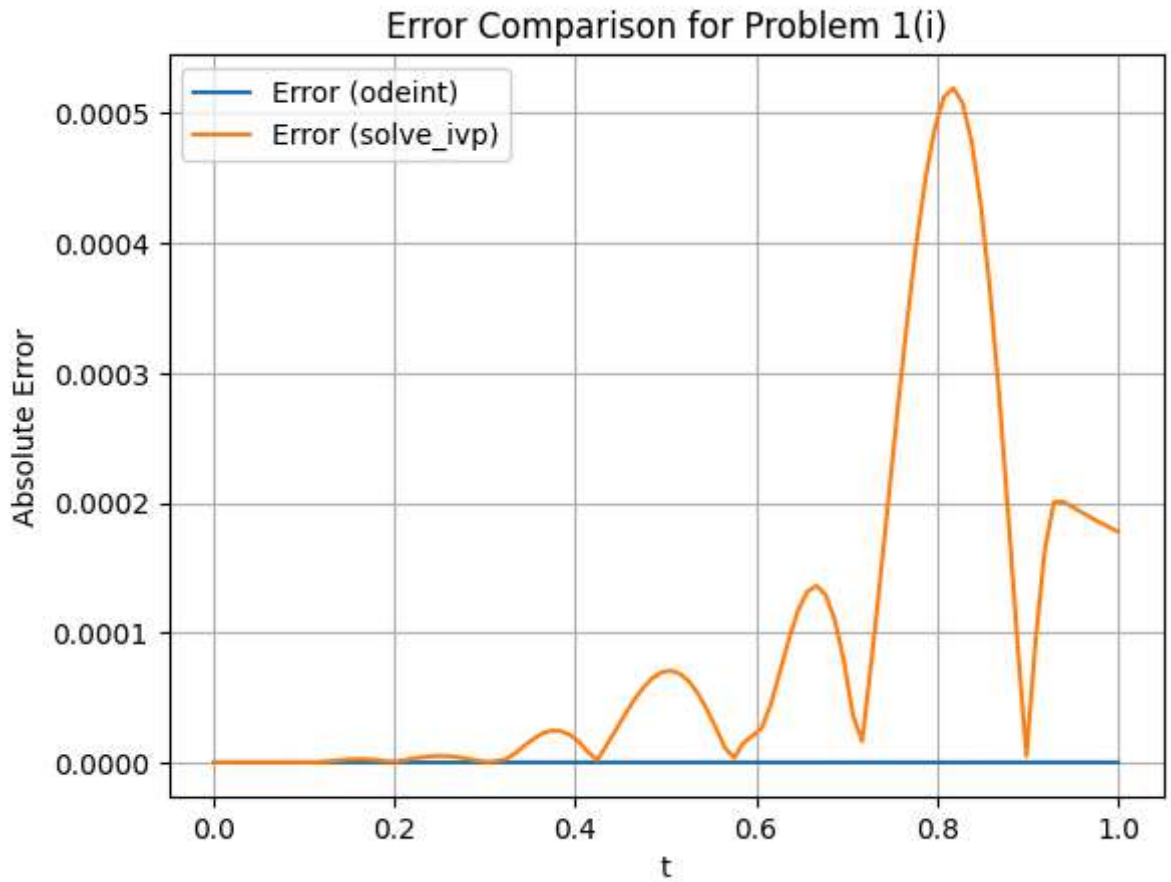
Assignment 03

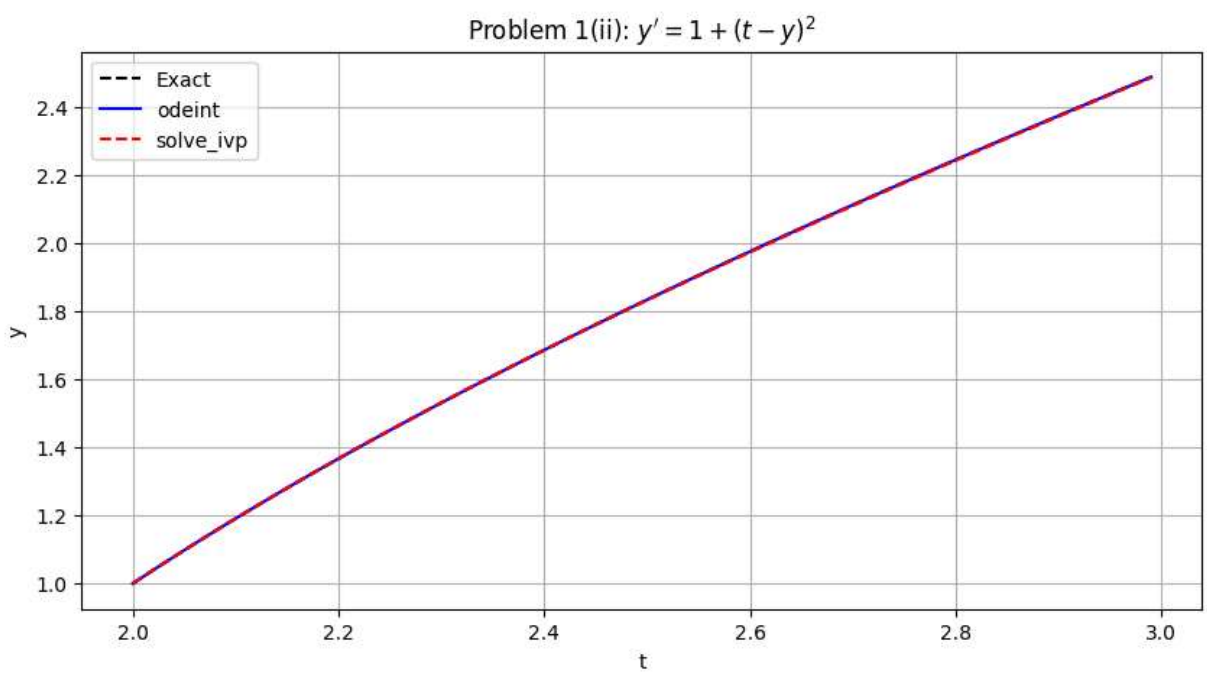
Question 01

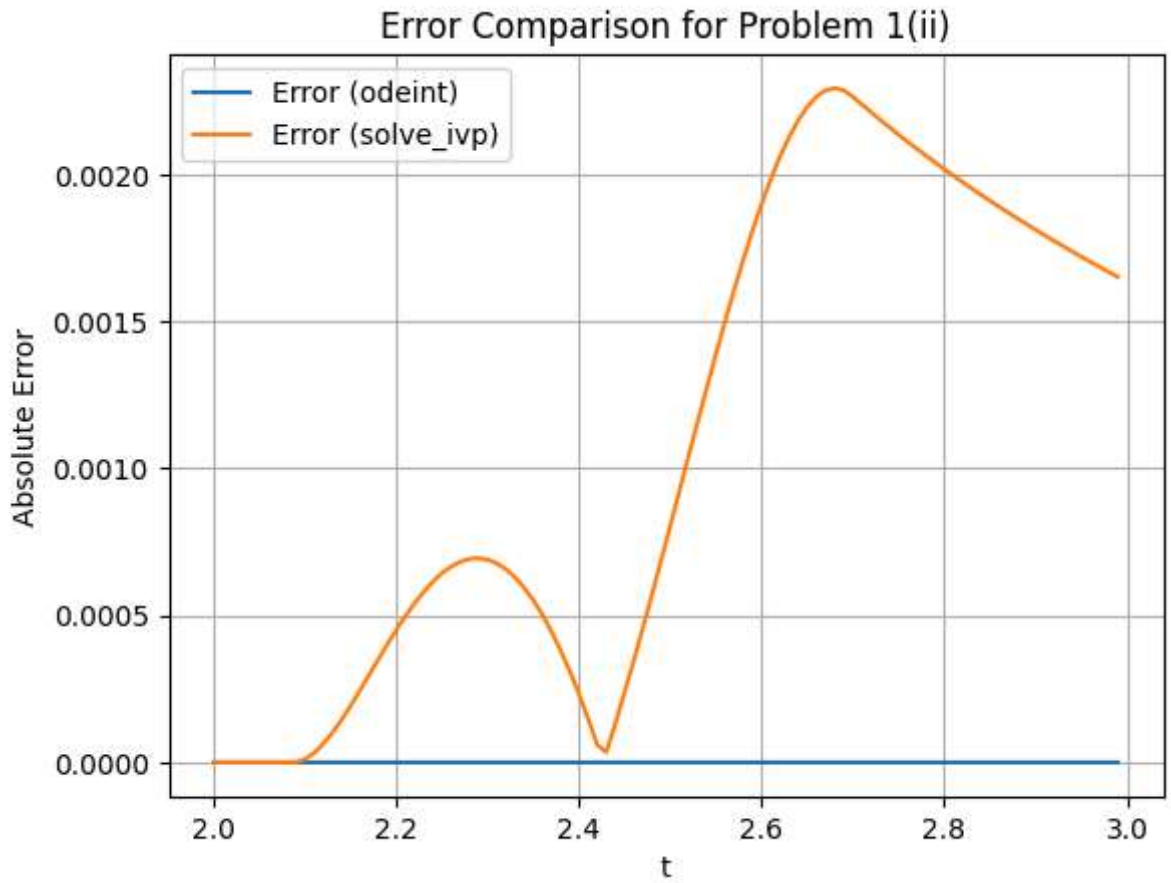
In [8]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import odeint,solve_ivp
# Problem 1(i)
def f1(y,t):
    return t*np.exp(3*t)-2*y
def exact1(t):
    return (1/5)*t*np.exp(3*t)-(1/25)*np.exp(3*t)+(1/25)*np.exp(-2*t)
t1=np.linspace(0,1,100)
y0_1=0
# Using odeint
sol_odeint_1=odeint(f1,y0_1,t1).flatten()
# Using solve_ivp
sol_solve_ivp_1=solve_ivp(lambda t,y: f1(y,t),[0,1],[y0_1],t_eval=t1).y[0]
# Exact solution
y_exact1=exact1(t1)
# Plotting Problem 1(i)
plt.figure(figsize=(10,5))
plt.plot(t1,y_exact1,'k--',label='Exact')
plt.plot(t1,sol_odeint_1,'b-',label='odeint')
plt.plot(t1,sol_solve_ivp_1,'r--',label='solve_ivp')
plt.title("Problem 1(i): $y'=te^{3t}-2y$")
plt.xlabel('t')
plt.ylabel('y')
plt.legend()
plt.grid(True)
plt.show()
# Error plot
plt.figure()
plt.plot(t1,np.abs(y_exact1-sol_odeint_1),label='Error (odeint)')
plt.plot(t1,np.abs(y_exact1-sol_solve_ivp_1),label='Error (solve_ivp)')
plt.title("Error Comparison for Problem 1(i)")
plt.xlabel("t")
plt.ylabel("Absolute Error")
plt.legend()
plt.grid(True)
plt.show()
# Problem 1(ii)
def f2(y,t):
    return 1+(t-y)**2
def exact2(t):
    return t+1/(1-t)
t2=np.linspace(2,2.99,100)
y0_2=1
# Using odeint
sol_odeint_2=odeint(f2,y0_2,t2).flatten()
# Using solve_ivp
sol_solve_ivp_2=solve_ivp(lambda t,y: f2(y,t),[2,2.99],[y0_2],t_eval=t2).y[0]
```

```
# Exact solution
y_exact2=exact2(t2)
# Plotting Problem 1(ii)
plt.figure(figsize=(10, 5))
plt.plot(t2,y_exact2,'k--',label='Exact')
plt.plot(t2,sol_odeint_2,'b-',label='odeint')
plt.plot(t2,sol_solve_ivp_2,'r--',label='solve_ivp')
plt.title("Problem 1(ii): $y'=1+(t-y)^2$")
plt.xlabel('t')
plt.ylabel('y')
plt.legend()
plt.grid(True)
plt.show()
# Error plot
plt.figure()
plt.plot(t2,np.abs(y_exact2-sol_odeint_2),label='Error (odeint)')
plt.plot(t2,np.abs(y_exact2-sol_solve_ivp_2),label='Error (solve_ivp)')
plt.title("Error Comparison for Problem 1(ii)")
plt.xlabel("t")
plt.ylabel("Absolute Error")
plt.legend()
plt.grid(True)
plt.show()
```




Error Comparison for Problem 1(i)
Error (odeint)
Error (solve_ivp)
Absolute Error
0.0000
0.0001
0.0002
0.0003
0.0004
0.0005
0.0
0.2
0.4
0.6
0.8
1.0
t

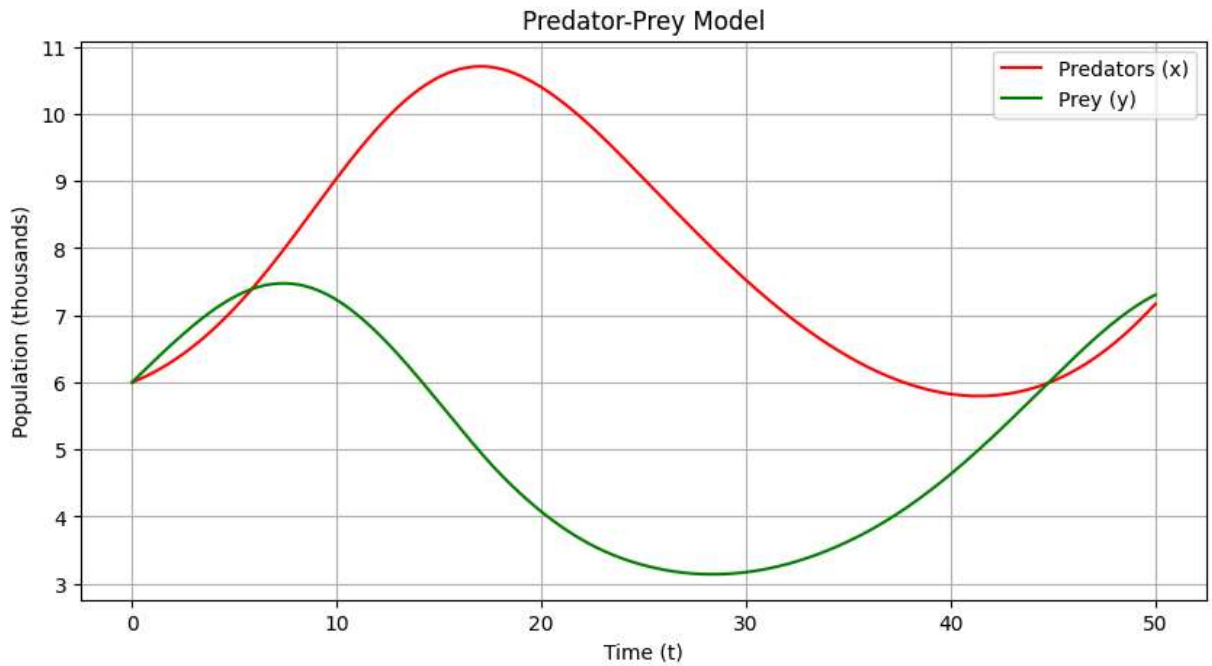

Problem 1(ii): $y' = 1 + (t - y)^2$
Exact
odeint
solve_ivp
y
1.0
1.2
1.4
1.6
1.8
2.0
2.2
2.4
2.0
2.2
2.4
2.6
2.8
3.0
t



Question 02

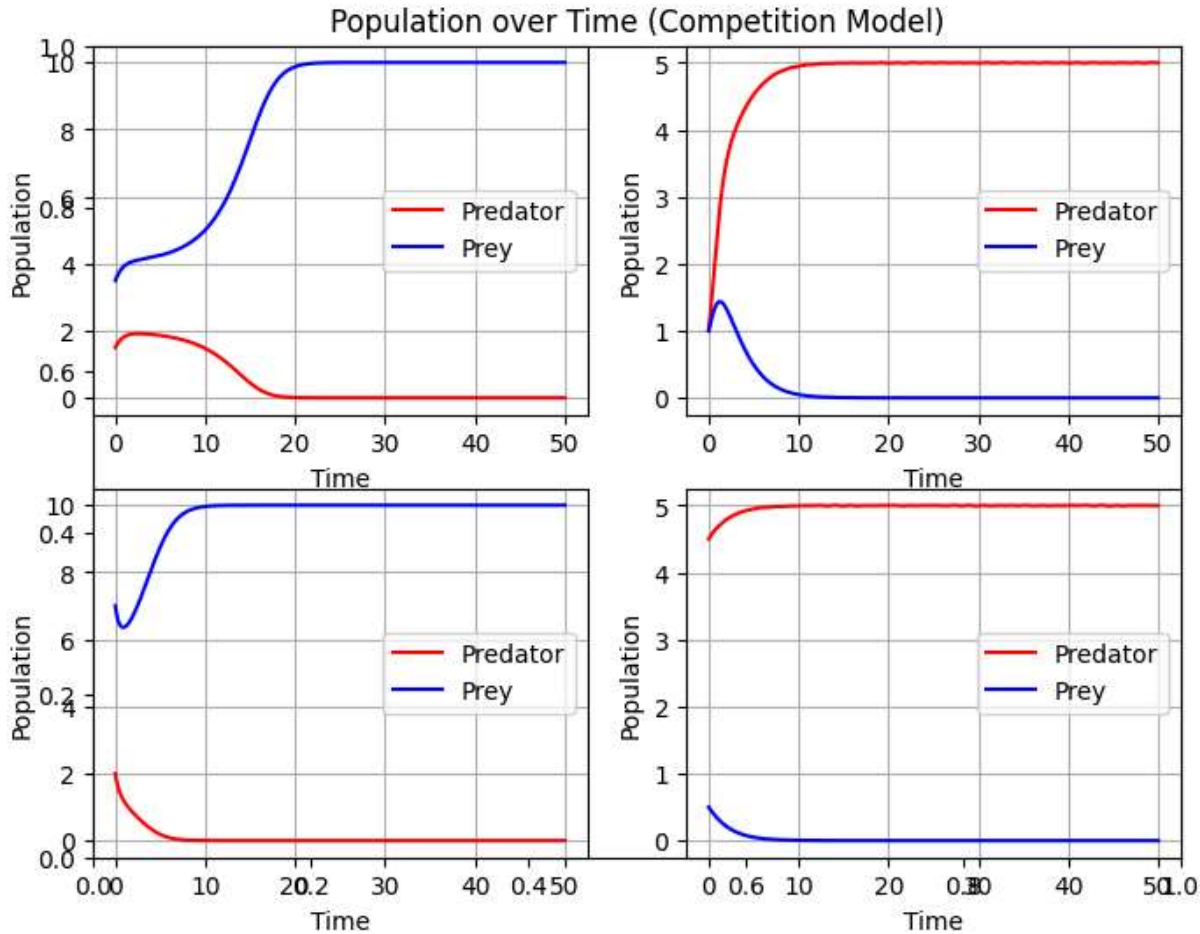
In [9]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
# Define the system
def predator_prey(t,z):
    x,y=z
    dxdt=-0.1*x+0.02*x*y
    dydt=0.2*y-0.025*x*y
    return [dxdt,dydt]
# Initial conditions
x0=6
y0=6
t_span=np.linspace(0,50,1000)
# Solve the system
sol=solve_ivp(predator_prey,[0,50],[x0,y0],t_eval=t_span)
t=sol.t
x=sol.y[0]
y=sol.y[1]
# Plot x(t) and y(t)
plt.figure(figsize=(10,5))
plt.plot(t,x,label='Predators (x)',color='red')
plt.plot(t,y,label='Prey (y)',color='green')
plt.title("Predator-Prey Model")
plt.xlabel("Time (t)")
plt.ylabel("Population (thousands)")
```

```
plt.legend()
plt.grid(True)
plt.show()
```



Question 03

In [10]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
# Define the competition model
def competition_model(t,z):
    x,y=z
    dxdt=x*(2-0.4*x-0.3*y)
    dydt=y*(1-0.1*y-0.3*x)
    return [dxdt,dydt]
# Initial conditions for the 4 cases
initial_conditions=[[1.5,3.5],[1,1],[2,7],[4.5, 0.5]]
t_span=np.linspace(0,50,1000)
plt.subplots(figsize=(8,6))
plt.title('Population over Time (Competition Model)')
# Solve and plot each case
for i, z0 in enumerate(initial_conditions):
    sol=solve_ivp(competition_model,[0,50], z0, t_eval=t_span)
    t=sol.t
    x,y=sol.y
    plt.subplot(2,2,i+1)
    plt.plot(t,x,label='Predator',color='red')
    plt.plot(t,y,label='Prey',color='blue')
    plt.xlabel('Time')
    plt.ylabel('Population')
    plt.legend()
    plt.grid(True)
plt.show()
```

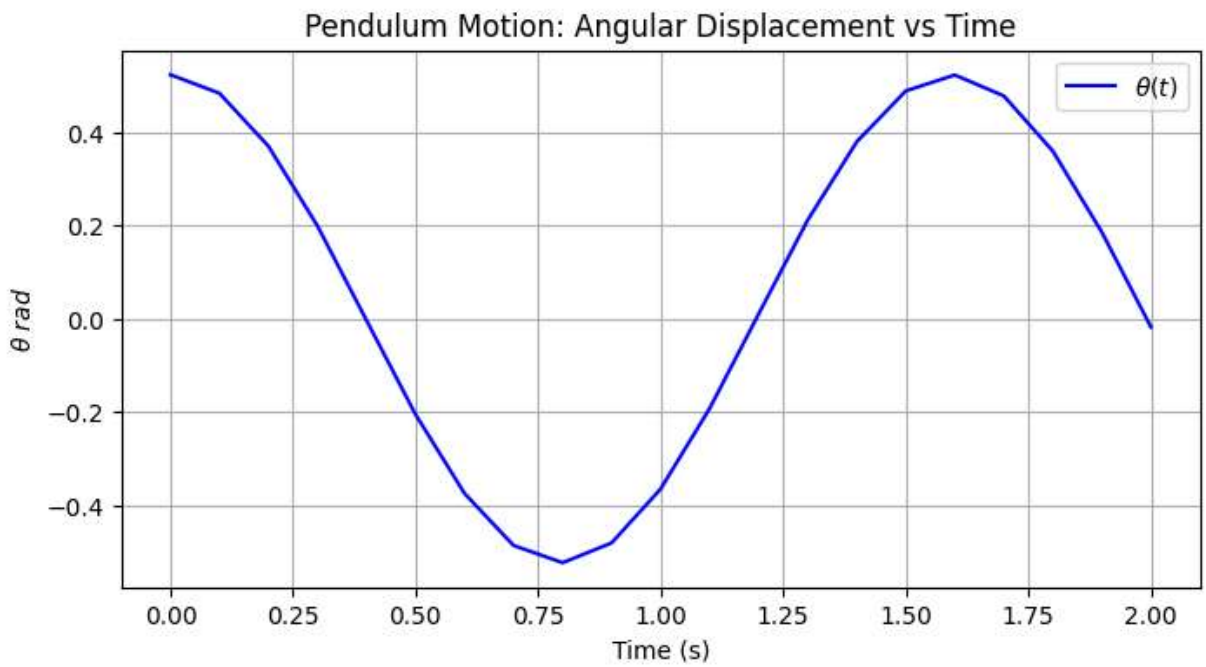


Question 04

In [11]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
def pendulum(t,z):
    theta,theta_dot=z
    dtheta_dt=theta_dot
    dtheta_dot_dt=-(32.17/2)*np.sin(theta)
    return [dtheta_dt,dtheta_dot_dt]
# Initial conditions
theta0=np.pi/6
theta_dot0=0
t_span=np.arange(0,2.1,0.1)
# Solve the system
sol=solve_ivp(pendulum, [0,2], [theta0,theta_dot0], t_eval=t_span,method='RK45')
t=sol.t
theta=sol.y[0]
# Display results
for i, j in zip(t,theta):
    print(f"t= {i:.1f}, theta= {j:.6f} rad")
# Plot theta vs time
plt.figure(figsize=(8,4))
plt.plot(t,theta,color='b',label=r'$\theta(t)$')
plt.title("Pendulum Motion: Angular Displacement vs Time")
plt.xlabel("Time (s)")
```

```
plt.ylabel(r'$\theta$ $rad$')
plt.grid(True)
plt.legend()
plt.show()
```

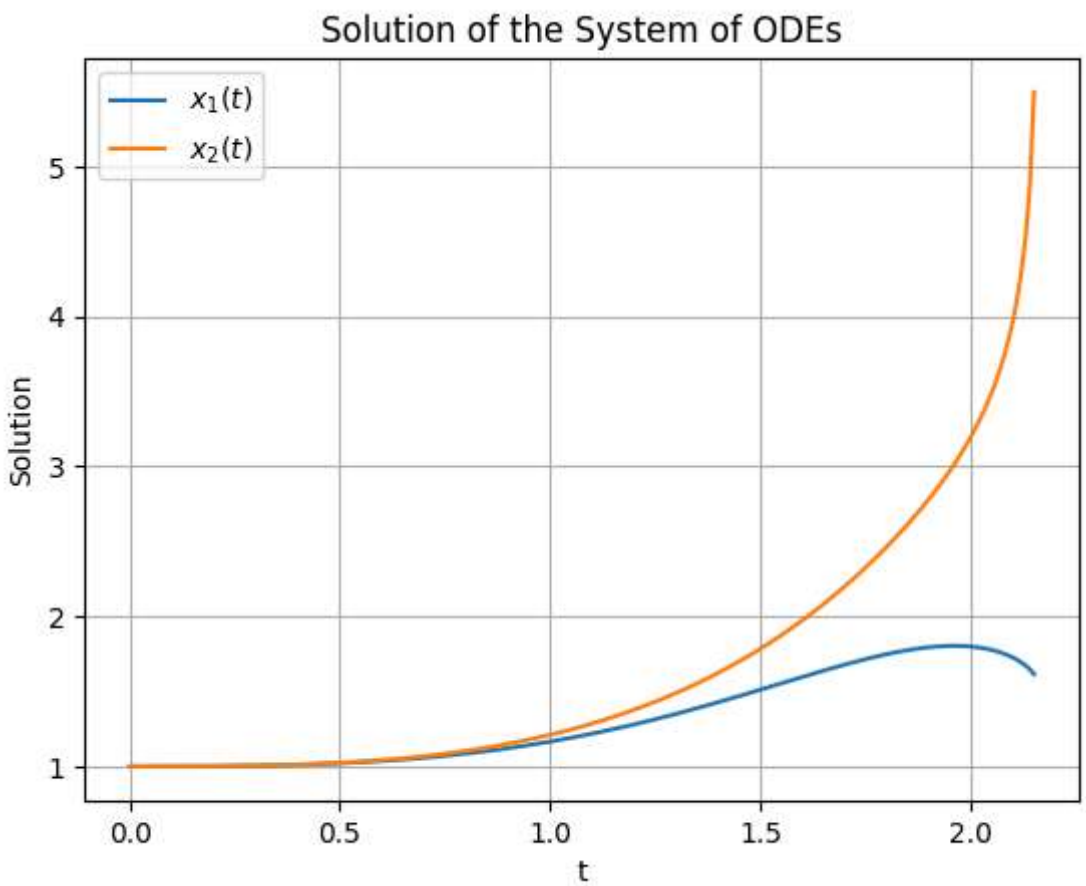
```
t= 0.0, theta= 0.523599 rad
t= 0.1, theta= 0.483854 rad
t= 0.2, theta= 0.370229 rad
t= 0.3, theta= 0.199187 rad
t= 0.4, theta= -0.003293 rad
t= 0.5, theta= -0.205173 rad
t= 0.6, theta= -0.374796 rad
t= 0.7, theta= -0.486320 rad
t= 0.8, theta= -0.523396 rad
t= 0.9, theta= -0.480898 rad
t= 1.0, theta= -0.365067 rad
t= 1.1, theta= -0.192304 rad
t= 1.2, theta= 0.010535 rad
t= 1.3, theta= 0.211690 rad
t= 1.4, theta= 0.379650 rad
t= 1.5, theta= 0.488745 rad
t= 1.6, theta= 0.522965 rad
t= 1.7, theta= 0.478040 rad
t= 1.8, theta= 0.359712 rad
t= 1.9, theta= 0.185532 rad
t= 2.0, theta= -0.017676 rad
```



Question 05

In [12]:
```
import numpy as np
from scipy.integrate import solve_ivp
import matplotlib.pyplot as plt
# Define the system of first-order ODEs
def system(t,y):
    y0,y1,y2,y3,y4,y5=y
    dy0=y1
```

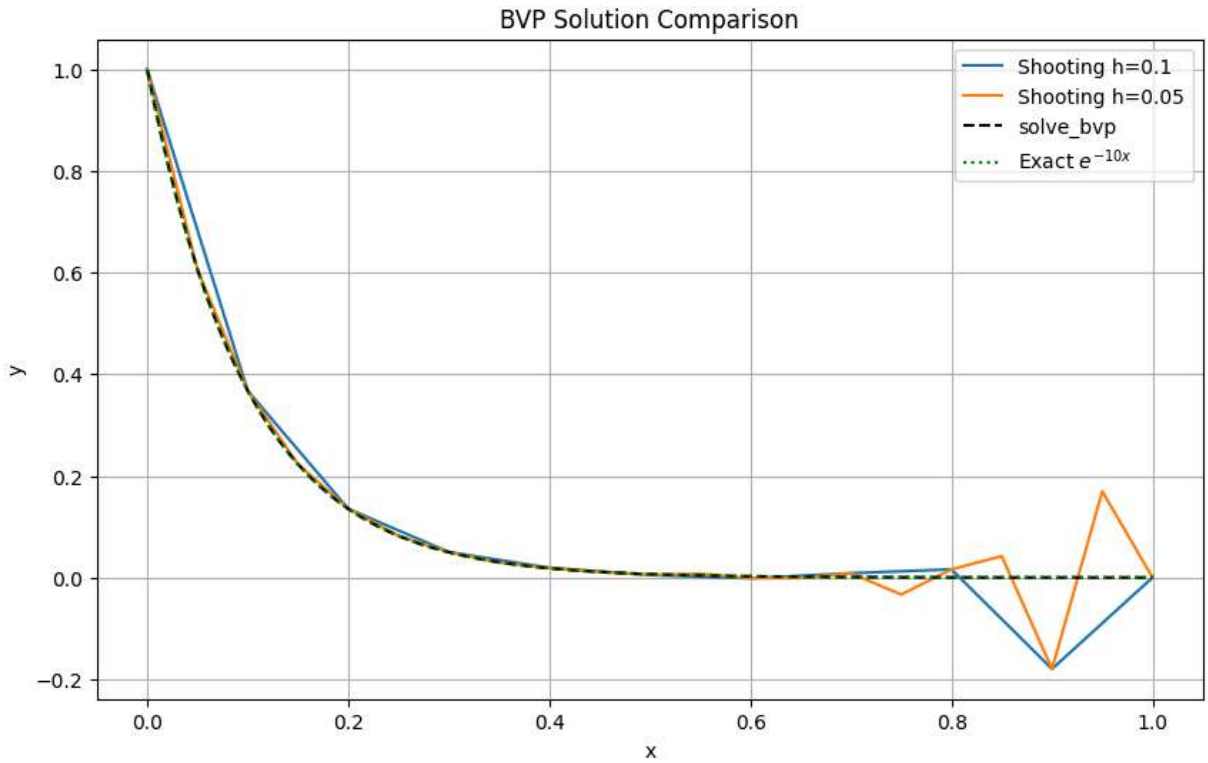
```
    dy1=y2
    dy2=-2*y4**2+y3
    dy3=y4
    dy4=y5
    dy5=-y2**3+y4+y0+np.sin(t)
    return [dy0,dy1,dy2,dy3,dy4,dy5]
# Initial conditions
y0_init=[1,0,0,1,0,0]
t_span=np.linspace(0,10,1000)
# Solve the system
sol=solve_ivp(system,[0,10], y0_init,t_eval=t_span)
# Plot the solutions
plt.plot(sol.t, sol.y[0],label='$x_1(t)$')
plt.plot(sol.t, sol.y[3],label='$x_2(t)$')
plt.xlabel('t')
plt.ylabel('Solution')
plt.title('Solution of the System of ODEs')
plt.legend()
plt.grid(True)
plt.show()
```



Question 06

In [13]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp,solve_bvp
# Define constants
a,b=0,1
```

```
ya,yb=1,np.exp(-10)
def ode_system(x,y):
    y1,y2=y
    dy1=y2
    dy2=100*y1
    return [dy1,dy2]
# Linear shooting method
def linear_shooting(h):
    N=int((b-a)/h)
    x_vals=np.linspace(a,b,N+1)
    # First IVP: y1'' = 100*y1, y1(0) = 1, y1'(0) = 0
    sol1=solve_ivp(ode_system,[a,b], [1,0], t_eval=x_vals)
    # Second IVP: y2'' = 100*y2, y2(0) = 0, y2'(0) = 1
    sol2=solve_ivp(ode_system,[a,b], [0,1], t_eval=x_vals)
    # Interpolate solution using shooting formula
    beta=yb
    y1b=sol1.y[0,-1]
    y2b=sol2.y[0,-1]
    c=(beta-y1b)/y2b
    y_approx=sol1.y[0]+c*sol2.y[0]
    plt.plot(x_vals,y_approx,label=f"Shooting h={h}")
    return x_vals, y_approx
# Solve using solve_bvp
def bvp_solution():
    def fun(x,y):
        return np.vstack((y[1], 100*y[0]))
    def bc(ya,yb):
        return np.array([ya[0]-1, yb[0]-np.exp(-10)])
    x=np.linspace(a,b,100)
    y_init=np.zeros((2, x.size))
    sol=solve_bvp(fun,bc, x, y_init)
    plt.plot(sol.x, sol.y[0],'k--',label='solve_bvp')
    return sol.x, sol.y[0]
# Plotting solutions
plt.figure(figsize=(10,6))
linear_shooting(0.1)
linear_shooting(0.05)
bvp_solution()
plt.plot(np.linspace(0,1,100),np.exp(-10*np.linspace(0,1,100)),'g:',label='Exact $e^{
plt.legend()
plt.xlabel('x')
plt.ylabel('y')
plt.title('BVP Solution Comparison')
plt.grid(True)
plt.show()
```


BVP Solution Comparison
Shooting h=0.1
Shooting h=0.05
solve_bvp
Exact e^{-10x}
y
x
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
0.0
0.2
0.4
0.6
0.8
1.0